Contractive Recurrent Classifiers on MNIST: Theory and Comparisons with Static and Adaptive Sparse Baselines

Alec Boyadjiev Boyd

April 27, 2026

Abstract. This report studies contractive recurrent classifiers (CRPs) on MNIST and compares them with feedforward MLP baselines in both static and adaptive sparse settings. We formalize the CRP normalization rule, prove contraction of the hidden-state map, derive a computable logit residual bound with a top-1 certification test, and show that feedforward MLP behavior can be embedded exactly in a feedforward CRP class. In a short c-sensitivity sweep, values near $c = 0.99$ retain most of the attainable accuracy while avoiding the large certification-step increase incurred as $c \to 1$. In the main six-way, hidden-node-matched comparison, static CRP variants are competitive with the MLP baseline, although the strongest random-sparse CRP condition is not edge-count matched and therefore does not isolate topology from edge budget. Under the reported 25-epoch schedule and severe shared DeepR budget, adaptive sparse models perform substantially worse and remain consistent with an underfitting regime; full-initialization adaptive CRP is the strongest adaptive condition, indicating potential for the approach.

Contents

1 Introduction

This report studies how a contractive recurrent classifier compares with a feedforward MLP baseline on MNIST, both in static settings and in adaptive sparse settings inspired by Deep Rewiring (DeepR). The main goal is to separate three issues that are often entangled: representational differences between feedforward and recurrent architectures, stability effects introduced by contraction, and optimization effects introduced by evolving sparse connectivity.

The repository used for this report exposes four base model families: `mlp`, `crp`, `mlp_adaptive`, and `crp_adaptive`. The accompanying experiment workflow also includes dedicated shortcut entry points for the CRP c-sensitivity sweep and the fixed six-condition comparison study; implementation-facing details about those runners, adaptive-budget configuration, and split policy are collected in Appendix A. The theoretical sections analyze the specified model classes and motivate the implementation choices used later in the report.

The report is organized as follows. Section 2 describes the model families and their adaptive variants. Section 3 develops the theoretical analysis for CRP-style contraction and certification. Section 4 reports the CRP contraction sensitivity experiment. Sections 5 and 6 present and interpret the six-way MLP/CRP comparison. Section 7 consolidates the principal limitations and future directions, and Appendix A records implementation-facing notes about model exposure, adaptive-budget configuration, seed/split behavior, and experiment shortcuts.

2 Models

We study two classifier families under a matched hidden-node-count comparison: a standard feedforward MLP baseline and a contractive recurrent classifier. We then evaluate adaptive sparse counterparts where connectivity is allowed to evolve during training using a Deep Rewiring-style mechanism [1]. In the interface used for this report, these correspond to four base model IDs: `mlp`, `crp`, `mlp_adaptive`, and `crp_adaptive`. The goal is not to maximize architectural novelty in each component, but to compare how recurrence, contraction, and evolving sparsity interact in a controlled setting.

2.1 Multi-Layer Perceptron (MLP) - Baseline

The baseline model is a conventional multi-layer perceptron trained with backpropagation, used here as a reference point for performance and optimization behavior. In our codebase, the MLP is purely feedforward, with configurable depth/width and nonlinear activations. Activations default to leaky ReLU, but standard ReLU is implemented as well. The model is initialized using a Kaiming Uniform initialization applied per linear weight matrix by default but can be initialized with standard linear initialization if preferred. This follows the classical supervised learning recipe that has remained the standard baseline for decades [8, 3].

At a high level, the baseline serves two purposes: (i) it provides a stable, well-understood comparison against recurrent dynamics, and (ii) it isolates the effect of structural adaptation by first measuring results in a static dense architecture. Any gains or trade-offs in the adaptive models can therefore be interpreted relative to a strong and interpretable control.

2.2 Contractive Recurrent Perceptron (CRP)

The CRP introduces recurrent hidden-state updates and explicitly biases the dynamics toward contraction. Conceptually, this places the model closer to stable recurrent systems, where bounded dynamics and attractivity are essential for learning. Our CRP design is not a direct reproduction of any single prior model; instead, it synthesizes ideas from several established lines of work [5, 7]. In the experiment workflow used for this report, CRP conditions include dense/base and feedforward structural variants as well as an implemented random-density variant used for the random-sparse CRP comparison later on.

The CRP uses three structured maps: input-to-hidden, hidden-to-hidden, and hidden-to-logit. In code, raw trainable tensors (no overline) are learned first, then masks/normalization are applied to produce effective matrices (overline) used in the recurrence. Concretely, with raw tensors W_{IH}, W_H, W_{HL} and fixed

masks M_{IH}, M_H, M_{HL}, we define

$$\begin{aligned}\overline{W}_{IH} &= W_{IH} \odot M_{IH},\\ \overline{W}_{HL} &= W_{HL} \odot M_{HL},\\ \overline{W}_H &= \text{Normalize}(W_H \odot M_H).\end{aligned}$$

The recurrent update therefore uses post-mask/post-normalization effective weights $\overline{W}$, not raw tensors W directly. The update rule is controlled by a memory parameter $\kappa \in (0, 1]$ and an iteration budget $t_{\max}$: larger κ puts more weight on the new nonlinear recurrent update, while $t_{\max}$ controls how long the model is allowed to settle. This gives a practical knob for trading off speed against convergence depth in inference. The exact update rule is

$$H_t = (1-\kappa)H_{t-1} + \kappa\,\phi\big(x\overline{W}_{IH} + H_{t-1}\overline{W}_H + b_H\big)\,,$$

where $H_0 = 0$. Contraction is enforced on the effective recurrent matrix $\overline{W}_H$. The first mode uses the classical ℓ_∞ operator norm (maximum absolute column sum). The second uses a weighted ℓ_∞ norm with a positive vector w, where each coordinate gain is measured as $m_j = ((|W_H \odot M_H|^\top w)_j / w_j)$ prior to final rescaling. The maximum coordinate gain $m = \max_j m_j$ is then used to determine the scaling factor $\min(1, c/m)$, so scaling is applied only when needed. Here, w approximates the dominant positive eigenvector direction of $|W_H \odot M_H|^\top$, computed by power iteration [2, 4]. Compared with plain ℓ_∞, this weighted norm can be tighter for anisotropic recurrent dynamics.

Under the mathematical specification analyzed in Section 3, the effective recurrent map satisfies $\|\overline{W}_H\|_{\infty,w} \le c < 1$ (or $\|\overline{W}_H\|_\infty \le c < 1$), so the hidden-state update has Lipschitz factor $\rho = (1-\kappa) + \kappa c < 1$, and the resulting CRP update is contractive (for $\kappa > 0$, with a 1-Lipschitz activation like ReLU/LeakyReLU) [6].

The model also includes a certification-style early-exit mechanism. At each recurrent step, the code forms a residual logit-error bound, given as

$$\Gamma_t = \|\overline{W}_{HL}^\top\|_\infty \frac{\rho}{1-\rho}\,\|H_t - H_{t-1}\|_\star, \quad \|\cdot\|_\star = \begin{cases}\|\cdot\|_{\infty,w}, & \text{weighted}\\ \|\cdot\|_\infty, & \text{plain}\end{cases}$$

and checks whether the winning logit is winning by more than double that bound (to account for potential change in both directions). If the top logit is tied, this margin test is not satisfied, so no certification is issued at that step (either tied class may pull ahead). Certified samples are then frozen via masking in subsequent steps. Computation continues for the batch up to $t_{\max}$, but certified samples no longer update.

Backpropagation then proceeds through the resulting masked unrolled graph in the standard way.

2.3 Adaptive Variants

The adaptive MLP and adaptive CRP keep the same high-level computational graphs as their non-adaptive counterparts, but replace static dense weights with sparse, budget-constrained connectivity that can change during learning. The inspiration is Deep Rewiring (DeepR): maintain a fixed parameter budget while allowing active connections to be pruned and regrown so the topology co-evolves with optimization [1]. In the reported experiments, these adaptive models reuse layer dimensions or structural templates from the non-adaptive baselines, but they are freshly initialized; they are not warm-started from separately trained dense MLP or CRP checkpoints.

Concretely, adaptation is implemented by replacing each trainable dense matrix with a DeepR-managed masked matrix module. Each module maintains (i) an immutable allowed-edge mask, (ii) a binary active mask A, (iii) a sign buffer $S \in \{-1, 0, +1\}$, and (iv) trainable nonnegative magnitudes θ. The effective weight used in forward passes is

$$W_{\text{eff}} = (A \odot \text{allowed}) \odot \big(S \odot \max(\theta, 0)\big),$$

so connectivity and weight values are both adaptive under a fixed edge budget.

In the adaptive MLP, this replacement is applied to every feedforward layer, with a single global budget across DeepR-enabled layers. In the adaptive CRP, the same mechanism is applied to raw tensors W_{IH}, W_H, W_{HL}, but DeepR participation can be toggled independently for the IH, HH, and HL blocks, so the most accurate wording is a global budget across DeepR-enabled matrices rather than automatically across every matrix in every configuration. This budget is configurable via `-k-total` and/or `-frac-total`; the value 10,000 discussed later is one comparison setting, not a universal constant. So the same distinction still holds: training updates raw state (no overline), while the recurrence uses effective matrices (overline) produced at forward time, i.e. $\overline{W}_{IH}, \overline{W}_H, \overline{W}_{HL}$. The base CRP dynamics, contraction normalization, and certification logic are preserved; only weight construction becomes adaptive. For CRP-adaptive, allowed edges can either follow the original structural masks or be expanded to full adjacency, depending on configuration, but they default to full adjacency; restricting the model to mask-only adjacency requires explicitly enabling the mask-adjacency option.

Training uses a two-part update: standard optimizer updates for ordinary parameters, followed by an explicit DeepR step for θ. For active edges, θ is updated by gradient, drift, and optional noise; edges with negative θ are pruned; then dormant allowed edges are regrown uniformly at random to restore the global target budget. Newly activated edges receive fresh random signs and magnitude zero (by default, but an option for seeded magnitudes is also implemented). To keep this separation explicit, DeepR θ parameters are excluded from the default optimizer parameter traversal and are updated only through the rewiring step.

Importantly, this adaptation does not alter the CRP contraction guarantee.

DeepR changes only how raw tensors W_{IH}, W_H, W_{HL} are parameterized and rewired; before each forward update, the effective recurrent matrix $\overline{W}_H$ is still normalized with the same rule (plain or weighted ℓ_∞) to enforce $\|\overline{W}_H\|_\star \leq c < 1$. Therefore the same per-step Lipschitz bound $\rho = (1 - \kappa) + \kappa c < 1$ applies, so contractivity is preserved (for $\kappa > 0$ and a 1-Lipschitz activation).

3 Theoretical Results for CRP Models

We first formalize feedforward MLPs via an MLP specification/class pair, then formalize CRP relative to a fixed CRP specification/class pair. We then state two guarantees that hold for every instance in a fixed CRP class: (i) contraction of hidden-state dynamics under the normalization rule, and (ii) a computable logit residual bound yielding a top-1 certification test. These results are mathematical analyses of the specified model class and motivate the implementation choices used later in the report; they are not presented as automated theorem checks executed during model training.

Definition 1 MLP specification

An MLP specification is a tuple

$$\xi = (\phi, L),$$

where $L \in \mathbb{N}_{\geq 1}$ is the number of hidden layers and ϕ is a coordinatewise activation.

This specification is intentionally minimal: only nonlinearity and depth are fixed at the specification level. As with CRP, exact tensor sizes are treated as instance-level properties, which keeps the formal statements architecture-focused rather than implementation-specific.

Definition 2 MLP class under an MLP specification

Fix an MLP specification ξ. The corresponding MLP class $\mathcal{M}_\xi$ consists of all tuples

$$\psi = (W_1, \ldots, W_L, W_{\text{out}}, b_1, \ldots, b_L, b_{\text{out}}),$$

with mutually compatible dimensions and a shared hidden width across all hidden layers. Equivalently, there exist inferred dimensions $d_0, d_{\text{hid}}, d_y \in \mathbb{N}_{\geq 1}$ (determined by ψ) such that

$$W_1 \in \mathbb{R}^{d_0 \times d_{\text{hid}}}, \qquad W_\ell \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{hid}}} \ (\ell = 2, \ldots, L),$$

$$b_\ell \in \mathbb{R}^{d_{\text{hid}}} \ (\ell = 1, \ldots, L), W_{\text{out}} \in \mathbb{R}^{d_{\text{hid}} \times d_y}, \qquad b_{\text{out}} \in \mathbb{R}^{d_y}.$$

For input $x \in \mathbb{R}^{d_0}$, define

$$h^{(0)} := x, \qquad h^{(\ell)} := \phi\big(h^{(\ell-1)} W_\ell + b_\ell\big), \ \ell = 1, \ldots, L,$$

$$z_{\text{MLP}} := h^{(L)} W_{\text{out}} + b_{\text{out}}.$$

The equal-hidden-width condition is the key modeling quirk for this section: it aligns the MLP with the feedforward block construction used later for CRP. Writing the class this way lets the embedding result quantify over all admissible parameter tuples $\psi \in \mathcal{M}_\xi$, not only a single trained network.

Definition 3 CRP specification

A CRP specification is a tuple

$$\eta = (\phi, \kappa, c, t_{\max}, \|\cdot\|_\star),$$

where ϕ is coordinatewise and 1-Lipschitz under $\|\cdot\|_\star$, $\kappa \in (0, 1]$, $c \in [0, 1)$, and $\|\cdot\|_\star$ is either $\|\cdot\|_\infty$ (plain mode) or $\|\cdot\|_{\infty,w}$ (weighted mode), and $t_{\max}$ is an integer.

Compared with the MLP specification, the CRP specification includes dynamical controls $(\kappa, t_{\max})$ and stability controls $(c, \|\cdot\|_\star)$ because these directly shape trajectory behavior, fixed-point contraction, and certification tightness.

Definition 4 CRP class under a CRP specification

Fix a CRP specification η. The corresponding CRP class $\mathcal{C}_\eta$ consists of all instance states

$$\theta = (M_{IH}, M_H, M_{HL}, W_{IH}, W_H, W_{HL}, b_H, b_L),$$

where the masks M_{IH}, M_H, M_{HL} are structural (non-trainable, instance-specific) and $W_{IH}, W_H, W_{HL}, b_H, b_L$ are the trainable tensors. Dimensions are required to be mutually compatible (e.g., M_H square, and hidden/output dimensions consistent across all blocks). For any instance, define effective matrices (overline) by

$$\overline{W}_{IH} = W_{IH} \odot M_{IH}, \qquad \overline{W}_{HL} = W_{HL} \odot M_{HL},$$

$$\overline{W}_H = \min(1, c/\widehat{m})\,(W_H \odot M_H),$$

where $\widehat{m}$ is the gain returned by the CRP normalization algorithm (plain or weighted mode).

For any instance $\theta \in \mathcal{C}_\eta$ and fixed input x, hidden dynamics are

$$H_t = (1-\kappa)H_{t-1} + \kappa\,\phi\big(x\overline{W}_{IH} + H_{t-1}\overline{W}_H + b_H\big), \qquad t \geq 1,$$

with initialization $H_0 = 0$, and logits $z_t = H_t\overline{W}_{HL} + b_L$.

The raw-versus-effective split is central: optimization updates raw tensors, but dynamics are defined only through masked/normalized effective tensors. This is exactly what makes hard contraction guarantees compatible with flexible parameterization and, later, adaptive rewiring.

▶ **Proposition 3.1** Contraction of CRP hidden-state map. *Fix a class $\mathcal{C}_\eta$ and an instance $\theta = (M_{IH}, M_H, M_{HL}, W_{IH}, W_H, W_{HL}, b_H, b_L) \in \mathcal{C}_\eta$. Let*

$$F_x(h) = (1-\kappa)h + \kappa\,\phi(x\overline{W}_{IH} + h\overline{W}_H + b_H).$$

Define the logit map

$$z(h) := h\overline{W}_{HL} + b_L.$$

For any logit vector $v \in \mathbb{R}^{d_y}$, let $v^{[1]} \geq v^{[2]}$ denote its largest and second-largest coordinates (counting multiplicity), and define

$$\mathrm{Top}(v) := \{i : \ v_i = v^{[1]}\}.$$

Then for all $h, h' \in \mathbb{R}^{d_h}$,

$$\|F_x(h) - F_x(h')\|_\star \leq \rho \, \|h - h'\|_\star, \qquad \rho = (1-\kappa) + \kappa c < 1.$$

Hence F_x is a contraction and has a unique fixed point $H_*(x)$.

Proof. Let $\widetilde{W}_H := W_H \odot M_H$ and $\overline{W}_H = \beta \widetilde{W}_H$ with $\beta = \min(1, c/\widehat{m})$ as in CRP normalization. In plain mode, $\widehat{m} = \max(\|\widetilde{W}_H\|_\infty, \varepsilon)$, so $\|\widetilde{W}_H\|_\infty \leq \widehat{m}$. In weighted mode, with $A := |\widetilde{W}_H|^\top$, finite-step power iteration yields some $w^{(K)} > 0$ and

$$\widehat{m} = \max\left(\max_j \frac{(Aw^{(K)})_j}{w_j^{(K)}}, \varepsilon \right),$$

hence $\|\widetilde{W}_H\|_{\infty, w^{(K)}} \leq \widehat{m}$. Therefore, in either mode,

$$\|\overline{W}_H\|_\star = \|\beta \widetilde{W}_H\|_\star \leq \beta \widehat{m} \leq c.$$

So exact Perron-vector convergence is unnecessary for safety: finite power-iteration output is sufficient for the bound above [2, 4].

Now, for any h, h',

$$\begin{aligned} \|F_x(h) - F_x(h')\|_\star &= \|(1-\kappa)(h-h') + \kappa\big(\phi(u) - \phi(v)\big)\|_\star \\ &\leq (1-\kappa)\|h - h'\|_\star + \kappa \|\phi(u) - \phi(v)\|_\star, \end{aligned}$$

where $u = x\overline{W}_{IH} + h\overline{W}_H + b_H$ and $v = x\overline{W}_{IH} + h'\overline{W}_H + b_H$. By the specification, ϕ is 1-Lipschitz under $\|\cdot\|_\star$, so

$$\|\phi(u) - \phi(v)\|_\star \leq \|u - v\|_\star = \|(h - h')\overline{W}_H\|_\star \leq c\|h - h'\|_\star.$$

Combining gives

$$\|F_x(h) - F_x(h')\|_\star \leq \big((1-\kappa) + \kappa c\big)\|h - h'\|_\star = \rho \|h - h'\|_\star.$$

Since $\kappa \in (0, 1]$ and $c \in [0, 1)$ by the CRP specification, we get $\rho < 1$; Banach's fixed-point theorem applies [6]. ◀

This contraction guarantee is the backbone for the rest of the section. It turns recurrent evaluation into a fixed-point problem with a unique attractor, which is what allows finite-time residual bounds and certified early stopping.

▶ **Proposition 3.2** Guaranteed logit residual bound and certification. *Fix the same CRP specification η, class $\mathcal{C}_\eta$, instance θ, and input x. Let ρ and the unique fixed point $H_*(x)$ be as in proposition 3.1. Define*

$$\Delta_t := \|H_t - H_{t-1}\|_\star, \qquad \Gamma_t := \|\overline{W}_{HL}^\top\|_\infty \frac{\rho}{1-\rho}\,\Delta_t.$$

Define logits and top-1/top-2 quantities by

$$z_t := H_t\overline{W}_{HL} + b_L, \qquad z_* := H_*\overline{W}_{HL} + b_L,$$

$$\mathrm{Top}(z_t) = \{i : \ (z_t)_i = z_t^{[1]}\}, \qquad \mathrm{Top}(z_*) = \{i : \ (z_*)_i = z_*^{[1]}\},$$

$$m_t := z_t^{[1]} - z_t^{[2]},$$

where $z_t^{[1]}, z_t^{[2]}$ count multiplicity, so a tie at the largest logit gives $m_t = 0$. Then:

1. *(Logit residual bound) $\|z_* - z_t\|_\infty \le \Gamma_t$.*
2. *(Top-1 invariance certificate) If the current margin m_t satisfies*

$$m_t > 2\Gamma_t,$$

then both top-label sets are singletons and equal:

$$|\mathrm{Top}(z_t)| = |\mathrm{Top}(z_*)| = 1, \qquad \mathrm{Top}(z_t) = \mathrm{Top}(z_*).$$

In particular, if the top logit is tied at step t, then $m_t = 0$ and this condition fails, so no top-1 certification is issued at that step (either tied class may pull ahead).

Proof. By Proposition 3.1, F_x is a ρ-contraction, $\rho < 1$, and H_* is its unique fixed point. Since $H_{k+1} = F_x(H_k)$,

$$\|H_{k+1} - H_k\|_\star \le \rho\|H_k - H_{k-1}\|_\star \le \rho^{k-t}\Delta_t \qquad (k \ge t).$$

Hence

$$\|H_* - H_t\|_\star \le \sum_{k=t}^{\infty} \|H_{k+1} - H_k\|_\star \le \sum_{j=0}^{\infty} \rho^{j+1}\Delta_t = \frac{\rho}{1-\rho}\Delta_t.$$

Now

$$z_* - z_t = (H_* - H_t)\overline{W}_{HL},$$

so

$$\|z_* - z_t\|_\infty \le \|\overline{W}_{HL}^\top\|_\infty \|H_* - H_t\|_\star \le \|\overline{W}_{HL}^\top\|_\infty \frac{\rho}{1-\rho}\,\Delta_t = \Gamma_t.$$

For weighted mode, this uses the implementation's normalized w (with $\max_j w_j = 1$), giving $\|v\|_\infty \le \|v\|_{\infty,w}$.

For the margin statement, assume $m_t > 2\Gamma_t$. Since $m_t > 0$, $\mathrm{Top}(z_t)$ is a singleton; write $\mathrm{Top}(z_t) = \{y\}$. Then for every $j \neq y$,

$$(z_t)_y - (z_t)_j \geq z_t^{[1]} - z_t^{[2]} = m_t.$$

From $\|z_* - z_t\|_\infty \leq \Gamma_t$, each coordinate shift is at most Γ_t, so

$$(z_*)_y - (z_*)_j \geq \big((z_t)_y - (z_t)_j\big) - 2\Gamma_t \geq m_t - 2\Gamma_t > 0.$$

Hence y strictly dominates every $j \neq y$ in z_*, so $\mathrm{Top}(z_*) = \{y\}$. Therefore $\mathrm{Top}(z_t) = \mathrm{Top}(z_*)$, and both are singletons. ◀

Operationally, this proposition explains when CRP can stop recurrent updates without risking a top-1 change: the current margin must dominate the residual envelope. The explicit tie handling is intentional—when the top class is tied, the method abstains from certification because either tied class may still pull ahead.

▶ **Proposition 3.3** Every DAG recurrent mask admits a contractive weighted norm**.** *Let $M_H \in \{0,1\}^{d_h \times d_h}$ be a recurrent mask whose directed graph is a DAG, and let $W_H \in \mathbb{R}^{d_h \times d_h}$ satisfy $W_H = W_H \odot M_H$. Then for every $c \in (0,1)$ there exists $w \in \mathbb{R}^{d_h}_{>0}$ such that*

$$\|W_H\|_{\infty,w} := \max_j \frac{(|W_H|^\top w)_j}{w_j} \leq c.$$

Hence in weighted mode one can take $\widehat{m} \leq c$, so CRP normalization may leave W_H unchanged.

Proof. Index nodes in a topological order $1, \ldots, d_h$ for M_H, so $(M_H)_{ij} \neq 0$ implies $i < j$. Because $W_H = W_H \odot M_H$, we also have $(W_H)_{ij} \neq 0 \Rightarrow i < j$. Choose

$$w_1 := 1, \qquad w_j := \frac{1}{c} \sum_{i<j} |(W_H)_{ij}| w_i + 1, \quad j = 2, \ldots, d_h.$$

Then $w_j > 0$ for all j. For $j = 1$, $(|W_H|^\top w)_1 = 0$. For $j \geq 2$,

$$\frac{(|W_H|^\top w)_j}{w_j} = \frac{\sum_i |(W_H)_{ij}| w_i}{w_j} = \frac{\sum_{i<j} |(W_H)_{ij}| w_i}{w_j} \leq c.$$

Taking the maximum over j gives $\|W_H\|_{\infty,w} \leq c$. ◀

This is the structural bridge to feedforward behavior: under a DAG mask, there exists a witness vector that makes the weighted gain already contractive, so in principle normalization need not modify W_H. In practice, however, the implemented gain is estimated numerically (finite iterations, finite precision, and safety floors), which can produce slight overestimation and therefore mild extra shrinkage even in near-feedforward regimes.

▶ **Corollary 3.4** Every feedforward MLP is realizable by a feedforward CRP**.** *Fix an MLP specification ξ and an MLP instance $\psi \in \mathcal{M}_\xi$. For every $c \in (0,1)$, there exists a feedforward CRP instance (same hidden-layer count L, same inferred hidden-layer width, and total hidden nodes equal to L times that inferred width, with $\kappa = 1$ and $t_{\max} \geq L+1$) such that:*

1. *$z_t(x) = z_{\mathrm{MLP}}(x)$ for all $t \geq L+1$,*
2. *its joined hidden-layer matrix satisfies $\|W_H\|_{\infty,w} \leq c$ for some $w > 0$, so normalization can leave it unchanged ($\overline{W}_H = W_H$).*

Proof. Let $\psi = (W_1, \ldots, W_L, W_{\mathrm{out}}, b_1, \ldots, b_L, b_{\mathrm{out}}) \in \mathcal{M}_\xi$, and let d_{hid} denote the inferred shared hidden width from ψ (e.g., $d_{\mathrm{hid}} = \dim b_1$). Set $d_h := Ld_{\mathrm{hid}}$ and partition $H_t = (H_t^{(1)}, \ldots, H_t^{(L)})$ with each block $H_t^{(\ell)} \in \mathbb{R}^{d_{\mathrm{hid}}}$. Construct CRP parameters by blocks:

$$W_{IH} = [W_1 \;\; 0 \;\; \cdots \;\; 0],$$

$$(W_H)_{\ell,\ell+1} = W_{\ell+1} \quad (\ell = 1, \ldots, L-1), \qquad (W_H)_{ab} = 0 \text{ otherwise},$$

and take M_H to be the corresponding feedforward block mask (same nonzero block pattern), so $W_H = W_H \odot M_H$ and M_H is a DAG mask.

$$b_H = (b_1, \ldots, b_L), \qquad W_{HL} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ W_{\mathrm{out}} \end{bmatrix}, \qquad b_L = b_{\mathrm{out}}.$$

With $\kappa = 1$ and $H_0 = 0$, block updates are

$$H_t^{(1)} = \phi(xW_1 + b_1), \qquad H_t^{(\ell)} = \phi\big(H_{t-1}^{(\ell-1)} W_\ell + b_\ell\big), \;\; \ell \geq 2.$$

By induction on ℓ, $H_t^{(\ell)} = h^{(\ell)}$ for all $t \geq \ell$. Hence for $t \geq L$,

$$z_t = H_t W_{HL} + b_L = h^{(L)} W_{\mathrm{out}} + b_{\mathrm{out}} = z_{\mathrm{MLP}}.$$

Therefore item 1 holds for all $t \geq L+1$, i.e., exactly the hidden-layer count plus one propagation stage. By construction, M_H is DAG and W_H respects M_H, so by proposition 3.3 there exists $w > 0$ with $\|W_H\|_{\infty,w} \leq c$. Therefore normalization can keep W_H unchanged. ◀

Taken together, the last proposition and this corollary show an exact theoretical containment statement: CRP can realize feedforward MLP behavior exactly. Empirically, the normalization routine can still induce small deviations because it uses conservative finite-step gain estimates rather than an exact witness. The next sections therefore compare static feedforward CRPs vs. MLPs, and then their adaptive variants, to test whether the strictly larger (inclusive) adaptive-CRP state space improves discovery, or whether the more constrained adaptive-MLP state space yields better optimization outcomes.

4 Practical Results for CRP Models

This section outlines a simple controlled experiment to measure sensitivity to the CRP contraction hyperparameter c before introducing broader comparisons. The sweep is used only to choose a reasonable comparison setting for the later MNIST experiments. The corresponding shortcut runner, `-run-crp-c-sensitivity`, is documented in Appendix A.1.

We use a CRP construction of 128 hidden nodes on the MNIST dataset with:

1. full input-to-hidden connectivity ($M_{IH} = \mathbf{1}$),
2. full hidden-to-logit connectivity ($M_{HL} = \mathbf{1}$),
3. a randomly generated hidden adjacency mask M_H with approximately 50% sparsity (about half of entries active).

The hidden adjacency mask is sampled per trial and kept fixed during that trial.

To study behavior in the high-retention regime, we sweep c over values increasingly close to 1, e.g.

$$c \in \{1 - 10^{-k} : \ k = 1, 2, \ldots, 10\}.$$

In the experiment setup used for this sweep, for each c value we ran 25 independent trials, each for 5 epochs. Different random seeds were used across trials, so the initializations and hidden masks changed from run to run; depending on the split-seed policy, the train/validation partition can also vary across runs (see Appendix A.3). The results are collected in Table 1 and Figure 1.

Table 1: CRP c-sensitivity results on MNIST (25 trials per c, 5 epochs per trial).

c	Validation mean (%)	Validation std (%)	Test mean (%)	Test std (%)	Test cert mean (τ)
$1 - 10^{-1}$	0.9711	0.0022	0.9724	0.0019	4.1009
$1 - 10^{-2}$	0.9716	0.0020	0.9731	0.0020	5.2140
$1 - 10^{-3}$	0.9716	0.0016	0.9732	0.0016	6.2716
$1 - 10^{-4}$	0.9714	0.0018	0.9730	0.0018	7.3273
$1 - 10^{-5}$	0.9704	0.0026	0.9729	0.0017	8.5373
$1 - 10^{-6}$	0.9708	0.0017	0.9729	0.0013	12.5393
$1 - 10^{-7}$	0.9707	0.0023	0.9726	0.0019	14.8671
$1 - 10^{-8}$	0.9714	0.0027	0.9727	0.0019	15.2137
$1 - 10^{-9}$	0.9706	0.0026	0.9732	0.0015	15.2202
$1 - 10^{-10}$	0.9707	0.0024	0.9728	0.0018	15.4818

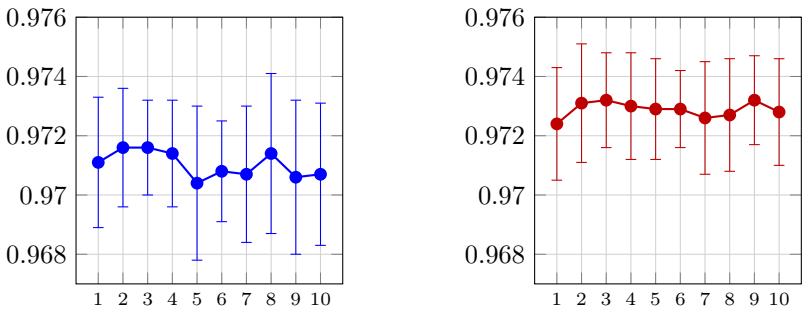

Figure 1: Shared c-sensitivity plot (left in blue: validation mean, right in red: test mean). X-axis ticks show k in $c = 1 - 10^{-k}$; both panels use the same y-range and one-standard-deviation error bars across 25 trials.

Audit checks found no evaluation-path mismatch or leakage, so this gap is not a code bug. The split behavior depends on seed policy: with a fixed split seed, the train/validation split is repeatable; with varying or unspecified split seeds, the validation split can change across runs. In the experiment setup used here, the validation set was not treated as one globally fixed benchmark across all runs, so some run-to-run variation can come from split variation as well as optimization noise. The fact that test exceeds validation in most runs therefore means that, for this sweep, the fixed official test split is slightly easier than the average validation benchmark actually used. A clean repair is to hold one stratified validation split fixed across all runs, with a split seed independent of the training seed, but we omit that change here because the gap is small (about +0.186 percentage points overall) and does not affect the c-sweep conclusion.

Table 1 and Figure 1 show that increasing c from 0.9 to 0.99 yields only a modest gain in mean test accuracy (0.9724 $\rightarrow$ 0.9731, an absolute increase of 0.0007). Beyond that, no reliable monotonic improvement is evident: for $c \geq 0.999$, test means remain in a narrow band (0.9726–0.9732), which is small relative to the reported trial-to-trial standard deviations. Figure 2 makes the tradeoff sharper: the mean accuracy-per-certification-step quantity falls from about 0.237 at $c = 0.9$ to about 0.063 at $c = 1 - 10^{-10}$, while Table 1 shows that the certification-step statistic itself rises from $\tau \approx 4.10$ to $\tau \approx 15.48$. Within this MNIST sweep, the practical conclusion is therefore limited and scoped: values around $c = 0.99$ capture most of the observed accuracy while avoiding the much larger recurrent-step cost incurred as $c \rightarrow 1$. A plausible interpretation is that MNIST offers limited benefit from substantially deeper recurrence under this short training schedule; datasets with richer long-range structure may behave differently.

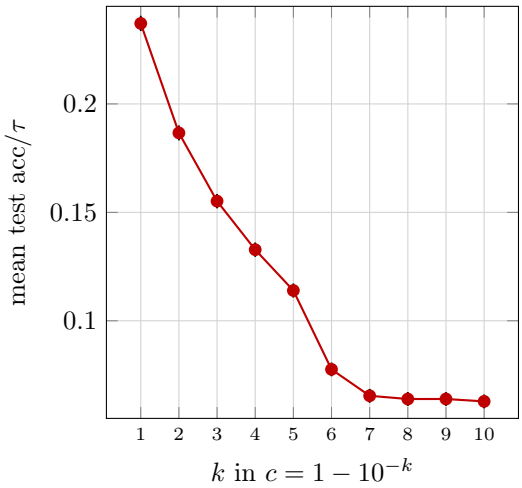

Figure 2: Each red point shows mean test accuracy divided by the mean certification step count τ at one setting $c = 1 - 10^{-k}$, so higher values mean better accuracy per recurrent step. As k increases (that is, as c moves closer to 1), this quantity drops sharply, showing that the extra recurrent steps grow much faster than the modest accuracy gains. Black vertical caps are one-standard-deviation error bars; they are tiny on this scale, but can be seen when zoomed in

The purpose of this experiment was to isolate whether c materially changes short-horizon training behavior in this partially sparse feedforward setting. The following sections extend that lens to direct static CRP-vs-MLP comparisons and then to adaptive variants, while the workflow details used to run those preset comparisons are summarized in Appendix A.1.

5 Comparison of CRP and MLP Models on MNIST

This section reports a controlled comparison among six model conditions on MNIST. Unless stated otherwise, every condition used the same training protocol:

- hidden nonlinearity: Leaky ReLU for both CRP and MLP,
- weight initialization: Kaiming initialization for all trainable weight matrices,
- matched model capacity by hidden-node count: 256 hidden nodes per model,
- MLP hidden layout: two feedforward hidden layers of $128 + 128$ nodes,
- statistical protocol: 25 independent trials per condition, each trained for 25 epochs,
- fairness control: all non-architecture settings (optimizer, learning-rate schedule, batch size, and preprocessing) were held fixed across all model conditions.

5.1 Conducted Model Conditions

We ran six model conditions under this shared protocol:

1. **Feedforward MLP.** We ran a standard feedforward MLP with two hidden layers of $128 + 128$ nodes.
2. **Random-sparse CRP.** We ran an implemented random-density CRP condition (`schematic=random_density`, used as the random-sparse baseline in the experiment workflow), whose hidden adjacency mask had approximately 50% active entries, sampled independently per trial and then held fixed.
3. **Feedforward-structured CRP.** We ran a CRP model constrained to a feedforward hidden adjacency pattern (DAG-style, aligned to a two-stage $128 + 128$ split).
4. **Adaptive MLP.** We ran an adaptive MLP with the same two-layer 128+128 feedforward layer dimensions as the dense baseline, but with fresh initialization and DeepR sparsity dynamics rather than warm-starting from a trained MLP.
5. **Adaptive CRP (feedforward initialization).** We ran an adaptive CRP with a feedforward structural initialization/configuration, not with pretrained weight transfer from a separately trained CRP.
6. **Adaptive CRP (full initialization).** We ran an adaptive CRP with a full-adjacency structural initialization/configuration, again without pretrained weight transfer from another model.

This setup enables a direct six-way comparison under a common training protocol, with the first three conditions serving as non-adaptive baselines and the last three as adaptive variants. The fixed preset identifiers exposed through `-run-comparison-condition` are summarized in Appendix A.1. The compar-

ison is matched by hidden-node count rather than by edge count. Under our counting convention—counting scalar entries in the input-to-hidden, hidden-to-hidden, and hidden-to-output blocks as edges, with expectations taken over the experiment's random-density construction—the two-layer feedforward MLP has 118,016 edges, whereas the random-sparse CRP has expected edge count 551,250. By contrast, the adaptive models in this comparison share a global budget across DeepR-enabled matrices, with $K_{\text{total}} = 10{,}000$ in the reported runs; in that setting, input-to-hidden, hidden-to-hidden, and hidden-to-output all participate. Appendix A.2 records the relevant configuration details. Each condition was run for 25 trials; Table 2 reports the final validation/test summary and Figure 3 shows the corresponding test-accuracy distributions.

Table 2: Validation/test comparison summary for the six model conditions (25 trials per condition, 25 epochs per trial).

Condition	Validation mean (%)	Validation std (%)	Test mean (%)	Test std (%)
MLP	0.9760	0.0018	0.9769	0.0021
R-CRP	0.9780	0.0019	0.9782	0.0017
FF-CRP	0.9757	0.0025	0.9762	0.0024
A-MLP	0.6178	0.0366	0.6214	0.0385
A-CRP-F	0.4394	0.0532	0.4430	0.0528
A-CRP-FC	0.6747	0.0221	0.6833	0.0269

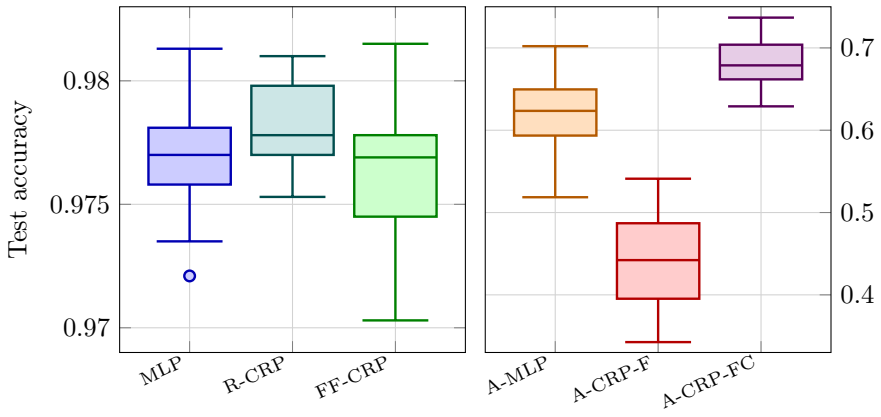

Figure 3: Test-accuracy boxplots for the six model conditions across 25 trials, with non-adaptive baselines on the left and adaptive variants on the right. Each panel uses its own readable y-range. Boxes show the interquartile range with the median, whiskers show the non-outlier range, and the single MLP outlier is plotted explicitly.

Table 2 shows that the three non-adaptive models all achieved mean test accuracy near 0.98, spanning only 0.9762 to 0.9782. Figure 3 is consistent with that narrow spread: the static boxplots have heavily overlapping interquartile

ranges, so the random-sparse CRP advantage is quantitatively small even before accounting for the unmatched edge budget. That ordering should therefore be interpreted cautiously, because the random-sparse CRP condition uses expected edge count 551,250 versus 118,016 for the MLP baseline under our counting convention. The adaptive conditions are qualitatively different in both level and spread. Table 2 gives mean test accuracies of 0.6833 (A-CRP-FC), 0.6214 (A-MLP), and 0.4430 (A-CRP-F), while Figure 3 shows clearly separated medians (roughly 0.6788, 0.6235, and 0.4422) and much wider variability than in the static panel. The adaptive standard deviations (0.0269, 0.0385, and 0.0528) are also an order of magnitude larger than those of the non-adaptive baselines, which is consistent with a much less stable training regime under the shared budget $K_{\text{total}} = 10{,}000$.

6 Discussion

What is empirically shown in the static regime. Table 2 and Figure 3 place all three non-adaptive models in essentially the same performance band: mean test accuracy ranges only from 0.9762 to 0.9782. Within this MNIST setup, the supported claim is therefore modest and scoped. Static CRP variants are competitive with a hidden-node-matched feedforward MLP baseline, but the present experiments do not show that recurrence is the unique cause of the small random-sparse CRP advantage, because that condition is not edge-count matched.

What is inferred from the feedforward CRP comparison. The near-match between the feedforward MLP and the feedforward-structured CRP is exactly the pattern one would expect from Corollary 3.4. The analysis in Section 3 shows that a feedforward MLP can be realized inside the feedforward CRP class at the specification level, so the tiny empirical gap between these two models is more naturally interpreted as an optimization or implementation effect than as an expressive failure. In particular, the containment result is existential, whereas the implemented model still uses numerical normalization of the recurrent block. As noted at the end of Section 3, finite-step gain estimation can introduce mild extra shrinkage even in nearly feedforward settings; that is a plausible explanation for why the feedforward CRP lands extremely close to, but slightly below, the matched MLP mean.

What is empirically shown in the adaptive regime. The adaptive results look much more like underfitting than overfitting. The run summaries underlying Table 2 show that the non-adaptive baselines finish with mean training losses on the order of 10^{-2}, whereas the adaptive MLP, adaptive CRP with feedforward initialization, and adaptive CRP with full initialization remain at roughly 1.55, 1.99, and 1.29, respectively, after the full 25-epoch schedule. Their validation and test accuracies track that weak training performance closely, which supports an underfitting diagnosis rather than a generalization failure after successful fitting. This poor adaptive performance should not be read as a failure of the model class itself: under a severe shared sparsity budget and a short training horizon, degradation of this kind is consistent with the general difficulty of optimizing sparse models in an undertrained regime.

Mechanistic inferences supported by the setup. A central inference is that the shared budget contributes materially to the adaptive difficulty. In the reported comparison, the adaptive models operate under $K_{\text{total}} = 10{,}000$ across the DeepR-enabled IH, HH, and HL blocks. Relative to the $784 \to 128 \to 128 \to 10$ feedforward layout, this is only $10{,}000/118{,}016 \approx 8.5\%$ of the corresponding dense edge count, and it is sparser still relative to the broader CRP search spaces. Because one budget is fragmented across matrices of very different sizes, many active edges are absorbed by the large input block, leaving fewer for recurrent and output communication. Regrowth dynamics also matter: newly activated edges begin with zero magnitude and must subsequently acquire useful

strength, so under a 25-epoch schedule optimization can spend substantial time reconstructing viable routes rather than refining an already competent sparse subnetwork. Appendix A.2 records these configuration details.

What remains conjectural. The ordering within the adaptive family is still informative, but the mechanistic explanation is not directly identified by the experiments. A reasonable conjecture is that, under a budget this severe, a fully recurrent initial graph is simply a better sparse starting point than a thinned feedforward witness because it offers more candidate routes for information flow that contraction can then stabilize. The gap between A-CRP-FC and A-CRP-F is therefore plausibly driven more by initialization quality than by a clean recurrence-versus-feedforward distinction, especially because adaptive CRP rewires over full adjacency by default unless mask-only mode is explicitly enabled (Appendix A.2).

Scope of the empirical claims. The results do support the claim that, on MNIST and under the reported training protocol, static contractive recurrent classifiers are competitive with matched-hidden-node MLP baselines. They do not support stronger causal claims about recurrence independent of edge count, nor do they imply that CRP is computationally cheaper: the present study does not perform compute-normalized benchmarking, and CRP requires repeated application of the hidden-state matrix at inference time. Likewise, the adaptive results identify the current training protocol as the dominant empirical bottleneck, but they do not by themselves establish limits of the broader adaptive model class.

7 Limitations and Future Work

This section makes explicit which conclusions are supported by the present evidence and which questions remain open.

7.1 Limitations

- **Short training horizon and hardware constraints.** The reported runs were performed in a CPU-only environment, which limited the practical training horizon: the c-sensitivity sweep used 5 epochs per trial, and the six-way comparison used 25 epochs per trial. Conclusions should therefore be read as statements about this short-horizon regime rather than about fully converged training.
- **Adaptive models remain undertrained.** The adaptive conditions finish with mean training losses around 1.55, 1.99, and 1.29, far above the non-adaptive baselines at roughly 10^{-2}. The adaptive results therefore reflect an underfitting regime under severe sparsity constraints, not a setting in which the adaptive models have clearly exhausted their optimization potential.
- **Static comparisons are not edge-count matched.** The static study intentionally matches hidden-node count, not edge count. Under the counting convention used here, the two-layer MLP has 118,016 edges, whereas the random-sparse CRP has expected edge count 551,250. As a result, the static comparison does not isolate recurrence from edge-budget effects.
- **No compute-normalized comparison is reported.** The study does not normalize by wall-clock time, recurrent-step count, FLOPs, or energy. Consequently, small accuracy differences between static conditions cannot be interpreted as efficiency gains, and the accuracy-versus-certification-step tradeoff in Section 4 should be read as a within-model diagnostic rather than a full compute comparison.
- **Validation split variability is present across runs.** Split behavior depends on seed policy; with varying or unspecified split seeds, the train/validation split can change across runs. This means the validation metric is not a single globally fixed benchmark across all trials. Appendix A.3 records this policy detail.
- **No confidence intervals or formal significance tests are included.** The report presents means and standard deviations across trials, but it does not report confidence intervals or hypothesis tests. The quantitative comparisons should therefore be interpreted descriptively rather than as formal statistical significance claims.

7.2 Future Work

The most direct next steps follow from those limitations.

- Extend the training schedules substantially, especially for the adaptive

conditions, to determine how much of the present gap is a short-horizon optimization effect.

- Replace the single global adaptive budget with per-matrix minimum budgets or other structured sparsity allocations across IH, HH, and HL so that recurrent and output pathways are not starved by the large input block.
- Use nonzero regrowth initialization or other seeded-regrowth strategies so that newly activated edges do not begin from zero magnitude.
- Add static comparisons that are edge-count matched in addition to hidden-node matched, so that topological effects can be separated more cleanly from raw representational budget.
- Add compute-aware benchmarking, such as wall-clock measurements or other cost-normalized comparisons, to evaluate whether any accuracy gain survives once recurrent evaluation cost is accounted for.
- Investigate improved rewiring strategies, including annealed rewiring, better regrowth proposals, and explicit comparisons between full-adjacency and mask-only rewiring.
- Fix one validation split across runs and supplement the descriptive summaries with confidence intervals or formal statistical tests to strengthen the inferential part of the comparison.

8 Conclusion

This report compared feedforward MLPs and contractive recurrent classifiers in both static and adaptive sparse settings on MNIST. The theoretical analysis justifies the CRP normalization rule and certification logic at the specification level, while the experiments show how those ideas behave under the reported short-horizon training protocols. Within this setup, static CRP variants are competitive with the matched hidden-node MLP baseline, but the random-sparse CRP advantage should be interpreted cautiously because the static comparison is not edge-count matched and is not compute normalized.

The adaptive results point to a different issue: under the reported 25-epoch schedule and shared $K_{\text{total}} = 10{,}000$ budget, the adaptive conditions remain in an underfitting regime, with full-initialization adaptive CRP emerging as the strongest adaptive configuration. The appropriate conclusion is therefore about the present training protocol, not about an intrinsic failure of the adaptive model class. Section 7 summarizes the main limitations of the current study and the most direct follow-up directions, while Appendix A records the implementation-facing workflow details used to generate the reported tables and figures.

A Appendix

This appendix records implementation-facing notes that clarify the interface assumed in the main text.

A.1 Repository and Shortcut Experiments

The repository used for this report exposes four base model identifiers:

- `mlp`,
- `crp`,
- `mlp_adaptive`,
- `crp_adaptive`.

In addition, the experimental workflow used for the reported sweeps provides two shortcut families:

- `-run-crp-c-sensitivity`, a dedicated entry point for the CRP c-sweep reported in Section 4;
- `-run-comparison-condition`, a fixed-condition entry point for the six-way comparison reported in Section 5.

The six fixed comparison-condition identifiers are:

- `mlp_feedforward`,
- `crp_random_sparse`,
- `crp_feedforward`,
- `mlp_adaptive`,
- `crp_adaptive_feedforward_init`,
- `crp_adaptive_full_init`.

These shortcut runners are treated as first-class experiment entry points in this report: they package preset configurations, support resumable checkpoints, and write CSV outputs for later aggregation. The random-sparse CRP condition is not a planned-only feature in this workflow; it is an implemented experiment path, used through the CRP option `schematic=random_density`.

A.2 Adaptive Configuration Notes

Adaptive MLP shares layer dimensions with the dense MLP baseline, and adaptive CRP shares structural templates with the corresponding CRP baselines, but the reported adaptive experiments do not rely on pretrained weight transfer from separately trained dense models. Instead, they begin from fresh initializations together with DeepR sparsity dynamics.

For CRP-adaptive, DeepR participation can be toggled independently on the IH, HH, and HL blocks (for example through `-deepr-ih`, `-deepr-hh`, and `-deepr-hl`). The budget should therefore be described as a global budget across DeepR-enabled matrices or layers, not automatically across every matrix

in every configuration. Budget selection is configurable via `-k-total` and/or `-frac-total`; the value 10,000 discussed in the main comparison is one experiment setting, not a universal constant. Allowed adjacency defaults to full adjacency, while restricting rewiring to the original structural masks requires explicitly using `-mask-adjacency-allowed`. Rewiring regrows uniformly at random from dormant allowed edges, and newly activated edges receive a fresh random sign and zero-magnitude θ by default.

A.3 Seed and Split Policy

The train/validation split behavior depends on seed policy. With a fixed split seed, the split is repeatable; with varying or unspecified split seeds, the split can vary across runs. Accordingly, statements in the main text about "different random seeds per trial" should be read as descriptions of the experiment runners used for the reported results, not as universal statements about every possible invocation of the training code.

References

[1] Guillaume Bellec et al. "Deep Rewiring: Training very sparse deep networks". In: *International Conference on Learning Representations (ICLR)*. 2018. URL: https://openreview.net/forum?id=BJ_wN01C-.

[2] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. 4th ed. Johns Hopkins University Press, 2013. URL: https://jhupbooks.press.jhu.edu/title/matrix-computations.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: https://www.deeplearningbook.org/.

[4] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. 2nd ed. Cambridge University Press, 2012. DOI: 10.1017/CBO9781139020411. URL: https://doi.org/10.1017/CBO9781139020411.

[5] Herbert Jaeger. *The "Echo State" Approach to Analysing and Training Recurrent Neural Networks*. Tech. rep. GMD Report 148. German National Research Center for Information Technology (GMD), 2001. URL: https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf.

[6] Erwin Kreyszig. *Introductory Functional Analysis with Applications*. Wiley, 1978. URL: https://onlinelibrary.wiley.com/doi/book/10.1002/9781118033104.

[7] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the difficulty of training recurrent neural networks". In: *International Conference on Machine Learning (ICML)*. 2013, pp. 1310–1318. URL: https://proceedings.mlr.press/v28/pascanu13.html.

[8] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0. URL: https://doi.org/10.1038/323533a0.